



گرافیک کامپیوٹری جلسہ دوم بخش دوم

حمیدرضا رضاپور

پر کردن ناحیه

پر کردن ناحیه، فرآیند رنگ آمیزی سطح یا ناحیه‌ی تصویری است.

ناحیه ممکن است در سطح پیکسل یا در سطح هندسی تعریف شود.

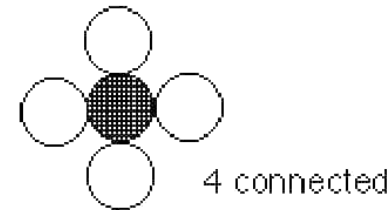
در سطح پیکسل، ناحیه را به دو روش توصیف می‌کنند: یکی **بر حسب پیکسل‌های مرزی محیط ناحیه** و دیگری **بر حسب تمام پیکسل‌های شامل آن**.

به الگوریتم‌هایی که برای روش اول به کار می‌روند، **الگوریتم‌های پر کردن مرزی** و به الگوریتم‌هایی که برای روش دوم به کار می‌روند، **الگوریتم‌های پر کردن سیلابی** می‌گویند.

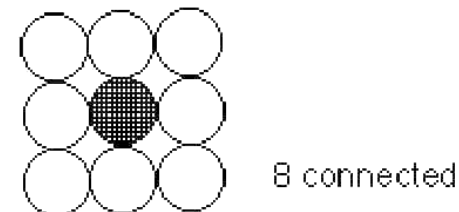
پر کردن ناحیه

اتصال‌های ۴ تایی و ۸ تایی

زمانی که یک طرح شامل پیکسل‌های مجزا در فضای تصویر باشد، دو راه برای اتصال چنین پیکسل‌هایی به یکدیگر برای تشکیل یک مرز پیوسته وجود دارد:



• اتصال ۴ تایی

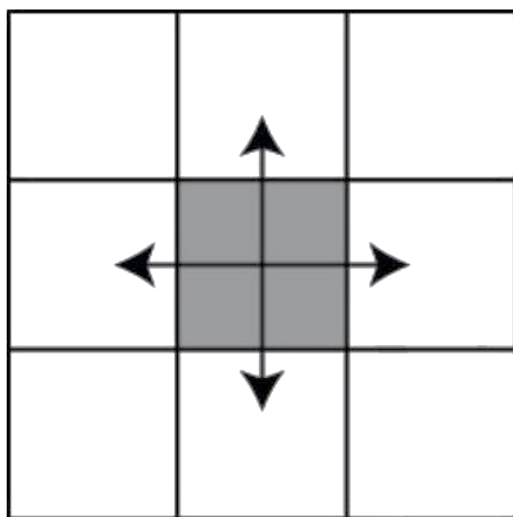


• اتصال ۸ تایی

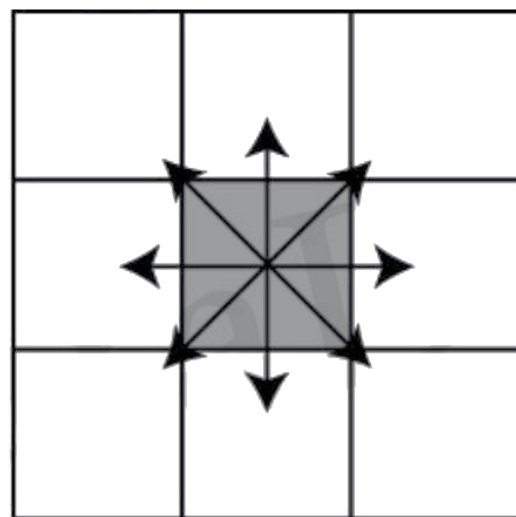
پر کردن ناحیه

اتصال‌های ۴ تایی و ۸ تایی

در اتصال ۴ تایی، برای یک پیکسل داده شده از ۴ طرف می‌توان به نواحی همسایه وارد شد: N, S, E, W .



4-Connectivity



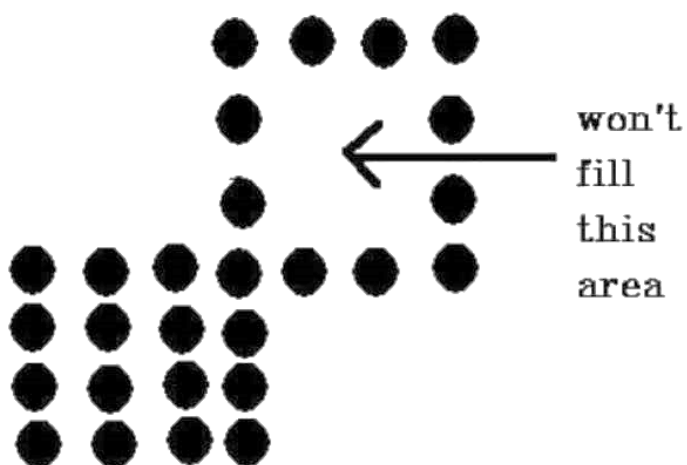
8-Connectivity

و در اتصال ۸ تایی، برای یک پیکسل داده شده از ۸ طرف می‌توان به نواحی همسایه وارد شد:

$N, S, E, W, NE, NW, SE, SW$

پر کردن ناحیه

نکته: اتصال ۴ تایی از اتصال ۸ تایی سریع تر است، اما همیشه نمی توان با تنها یک اتصال ۴ تایی ناحیه‌ی مورد نظر را پر کرد و شاید نیاز به ۲ اتصال ۴ تایی باشد که در این حالت واضح است که استفاده از یک اتصال ۸ تایی می تواند بهتر باشد.



پر کردن ناحیه

Region filling { Boundary-fill Algorithm
Flood-fill Algorithm
Scan-line Algorithm

پر کردن ناحیه

الگوریتم پر کردن مرزی

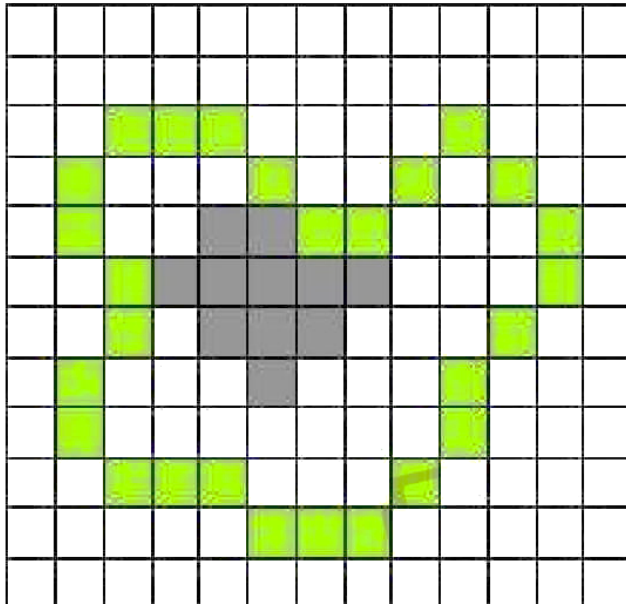
این الگوریتم با یک پیکسل ابتدایی داخل ناحیه شروع می کند و که آن را اصطلاحاً **دانه** می نامند و در هر مرحله چک می کند که آیا دانه ی ذکر شده یک پیکسل مرزی است یا قبلاً پر شده است.

اگر دانه ی مورد نظر پیکسل مرزی نباشد، پیکسل را پر می کند و از هر پیکسل همسایه به عنوان دانه ی جدید استفاده می کند.

اگر دانه ی مورد نظر مرزی باشد به شروع حلقه رفته و به فراخوانی هر پیکسل همسایه می پردازد.

پر کردن ناحیه

الگوریتم پر کردن مرزی

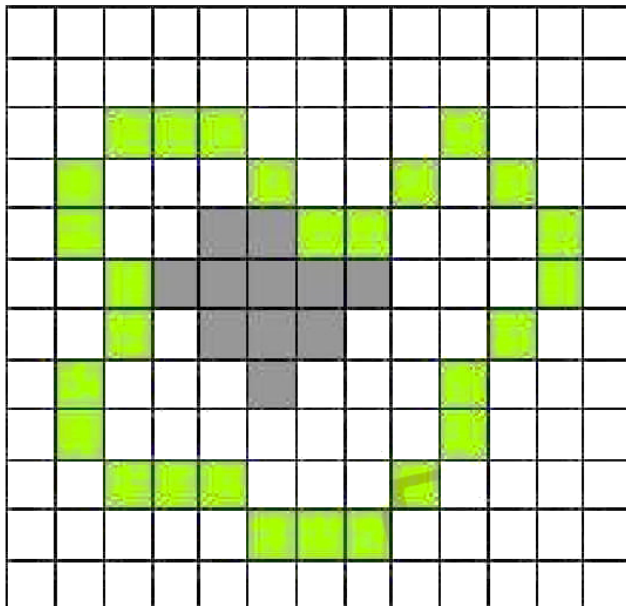


بنابراین به طور خلاصه روند کار به این صورت است:

- فرض می‌کنیم که مرز ناحیه‌ی مورد نظر با یک رنگ، رنگ شده باشد (رنگ مرزی).
- فرض می‌کنیم که قرار است پیکسل‌های داخل ناحیه با رنگی متفاوت از رنگ مرزی رنگ شوند (رنگ داخلی).

پر کردن ناحیه

الگوریتم پر کردن مرزی



بنابراین به طور خلاصه روند کار به این صورت است:

- با یک پیکسل داخل ناحیه شروع می‌کنیم که اگر مرزی نباشد و قبلاً با رنگ داخلی رنگ نشده باشد آن را رنگ می‌کنیم تا زمانی که تمام پیکسل‌های داخلی رنگ شوند.

پر کردن ناحیه

الگوریتم پر کردن مرزی

کد مربوط به این الگوریتم را در زیر مشاهده می کنید:

```
Void boundryFill (int x, int y, int fillColor, int borderColor){  
    int interiorColor;  
    getPixel (x, y, interiorColor);  
    if ((interiorColor != borderColor) && (interiorColor != fillColor)) {  
        setPixel (x, y, fillColor);  
        boundryFill (x + 1, y, fillColor, borderColor);  
        boundryFill (x - 1, y, fillColor, borderColor);  
        boundryFill (x, y + 1, fillColor, borderColor);  
        boundryFill (x, y - 1, fillColor, borderColor);  
    }  
}
```

پر کردن ناحیه

الگوریتم پر کردن مرزی

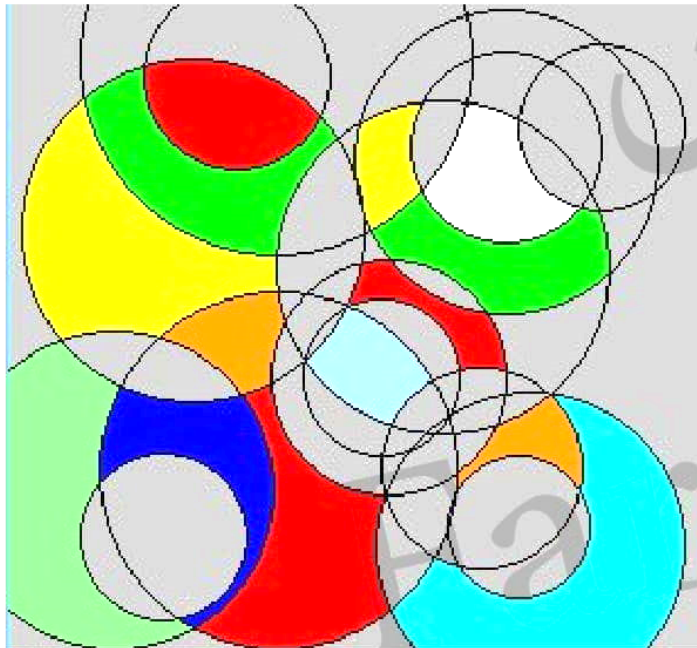
در این الگوریتم تعداد حلقه‌های بازگشتی زیاد است، به خصوص اگر ناحیه وسیع باشد.

اما راه‌هایی برای کم کردن تعداد حلقه‌های بازگشتی وجود دارد.

به عنوان مثال می‌توان ابتدا پیکسل‌های راست و چپ دانه‌ی مورد نظر را روی یک خط تا رسیدن به پیکسل مرزی پر کرد و سپس هر پیکسل بالا و پایین خطی که اسکن شده است را بررسی نمود که ببینیم آیا می‌توان از آن به عنوان داده‌ی جدید برای پر کردن خط افقی بعدی استفاده نمود یا خیر.

پر کردن ناحیه

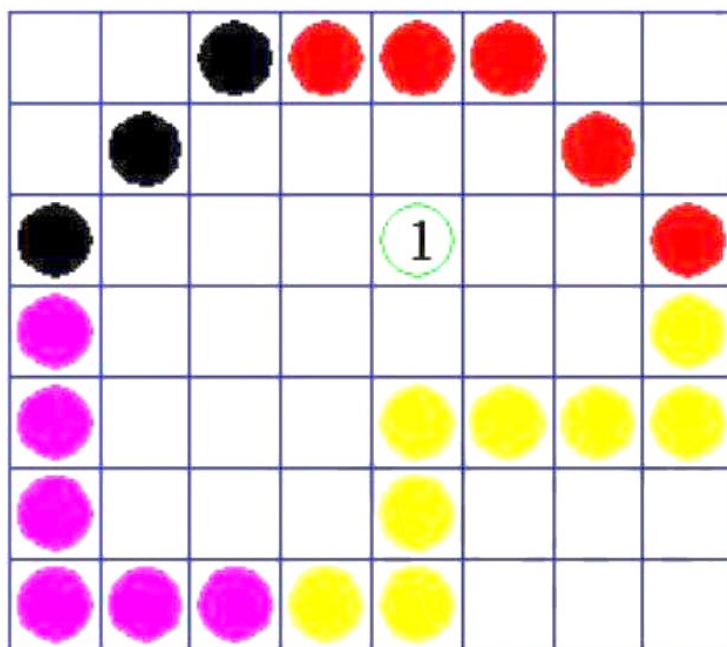
الگوریتم پر کردن سیلابی



در بعضی مواقع می‌خواهیم ناحیه‌ای که دارای مرز
چندرنگی است را پر کنیم و یا رنگش را عوض کنیم.
برای چنین ناحیه‌ای می‌توان به جای جستجو برای
رنگ مرز، از یک رنگ جایگزین استفاده نمود.

پر کردن ناحیه

الگوریتم پر کردن سیلابی



با یک پیکسل درون ناحیه شروع کرده و این الگوریتم بررسی می‌کند که آیا پیکسل مذکور دارای رنگ اصلی ناحیه است یا خیر.

اگر دارای رنگ اصلی ناحیه باشد، رنگ پیکسل مورد نظر را با رنگ جایگزین عوض می‌کند و از هر پیکسل همسایه به عنوان دانه‌ی جدید استفاده می‌کنیم.

اگر دارای رنگ اصلی ناحیه نباشد، به شروع حلقه‌ی فراخوانی رفته و از پیکسل‌های همسایه استفاده می‌شود.

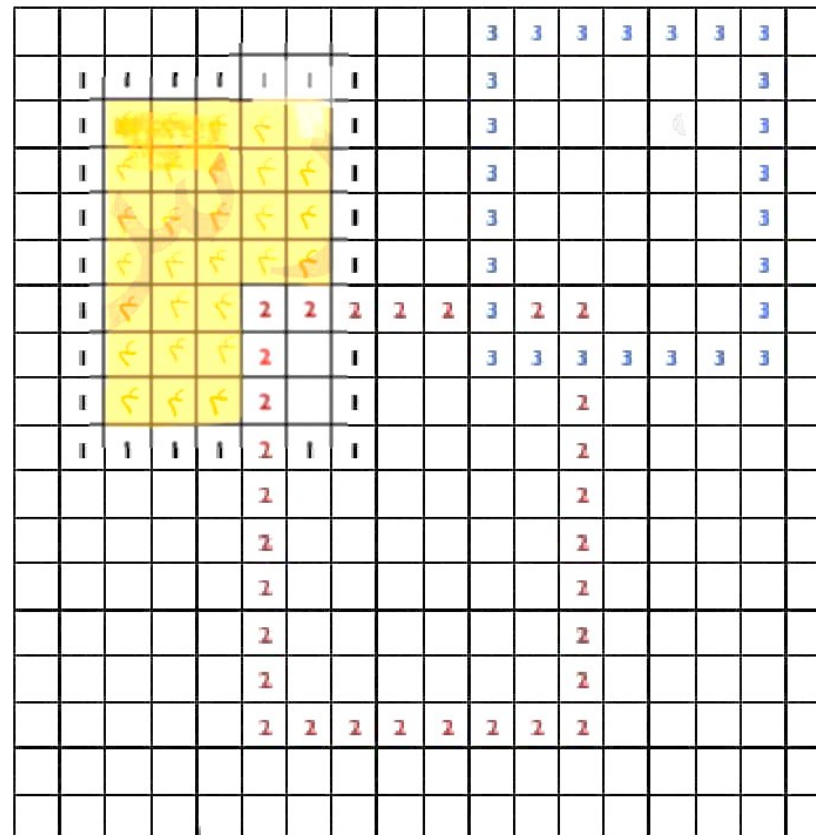
پر کردن ناحیه

											3	3	3	3	3	3	3	
											3						3	
											3						3	
											3						3	
											3						3	
											3						3	
					2	2	2	2	2	3	2	2					3	
				2						3	3	3	3	3	3	3	3	
				2								2						
					2							2						
				2								2						
				2								2						
				2								2						
				2								2						
				2								2						
				2	2	2	2	2	2	2	2	2	2					

پر کردن ناحیه

											3	3	3	3	3	3	3	
											3						3	
		↘	↘	↘	↘	↘					3						3	
		↘	↘	↘	↘	↘					3						3	
		↘	↘	↘	↘	↘					3						3	
		↘	↘	↘	↘	↘					3						3	
		↘	↘	↘	2	2	2	2	2	3	2	2					3	
		↘	↘	↘	2					3	3	3	3	3	3	3	3	
		↘	↘	↘	2							2						
					2							2						
					2							2						
					2							2						
					2							2						
					2							2						
					2							2						
					2	2	2	2	2	2	2	2	2					

پُر کردن ناحیه



پر کردن ناحیه

الگوریتم پر کردن سیلابی

کد مربوط به این الگوریتم را در زیر مشاهده می کنید:

```
Void floodFill (int x, int y, int fillColor, int interiorColor){  
    int Color;  
    getPixel (x, y, Color);  
    if (Color == interiorColor) {  
        setPixel (x, y, fillColor);  
        floodFill (x + 1, y, fillColor, interiorColor);  
        floodFill (x - 1, y, fillColor, interiorColor);  
        floodFill (x, y + 1, fillColor, interiorColor);  
        floodFill (x, y - 1, fillColor, interiorColor);  
    }  
}
```

پر کردن ناحیه

نکته:

تفاوت بین دو الگوریتم فوق در این است که الگوریتم پر کردن مرزی، مرز را با یک رنگ پر می‌کند. ولی الگوریتم پر کردن سیلابی، برای مرزهای چند رنگی استفاده می‌شود.

پر کردن ناحیه

الگوریتم خط اسکن

برخلاف دو الگوریتم قبلی که نواحی که در سطح پیکسل در فضای تصویر تعریف شده‌اند را پر می‌کنند، این الگوریتم، نواحی چندضلعی را که به صورت هندسی توسط مختصات رئوس تعریف شده‌اند پر می‌کند.

پر کردن ناحیه

الگوریتم خط اسکن

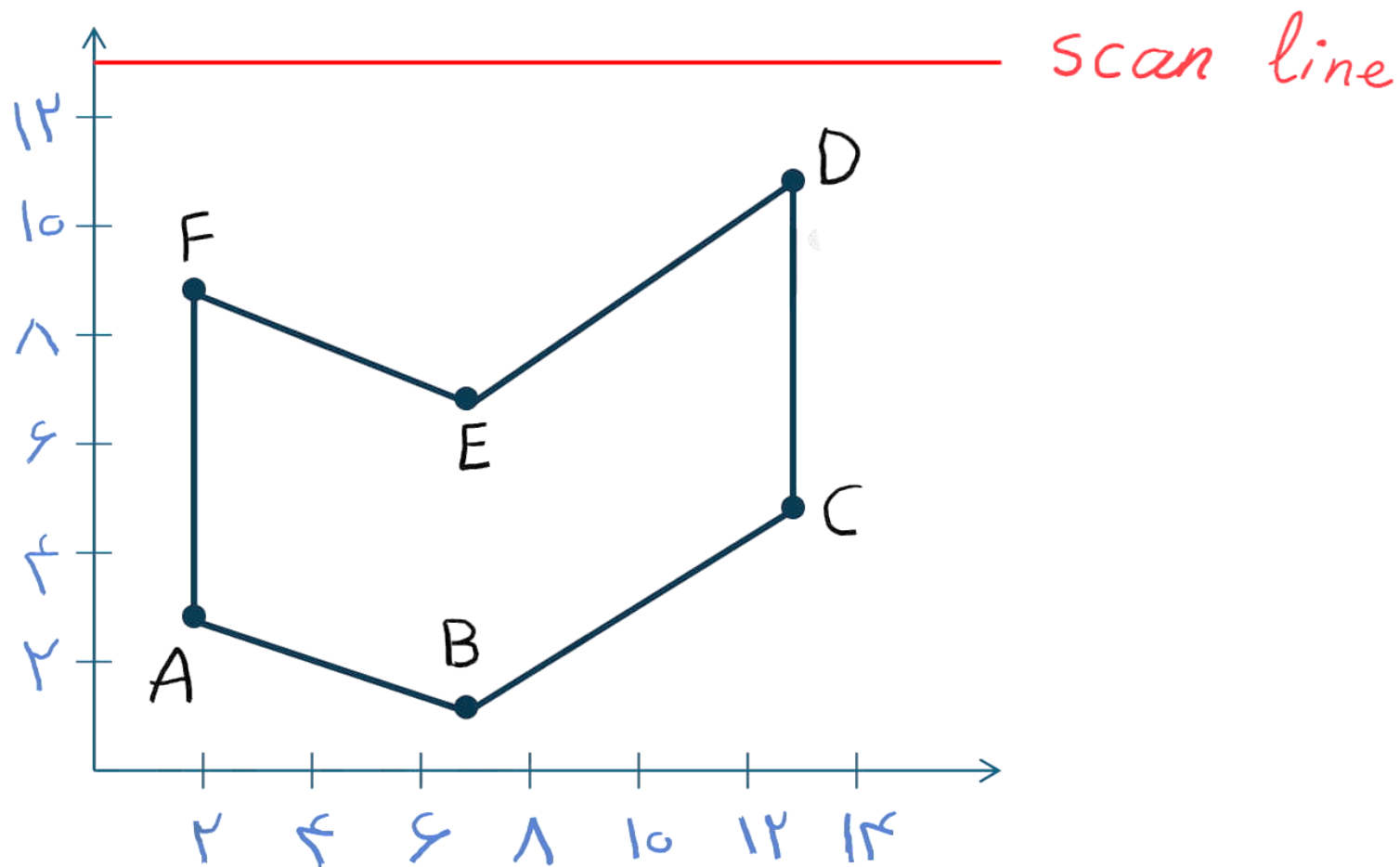
روند این الگوریتم بصورت زیر است:

تمام نقاط تقاطع خط اسکن را با یال‌های چندضلعی محاسبه می‌کند.

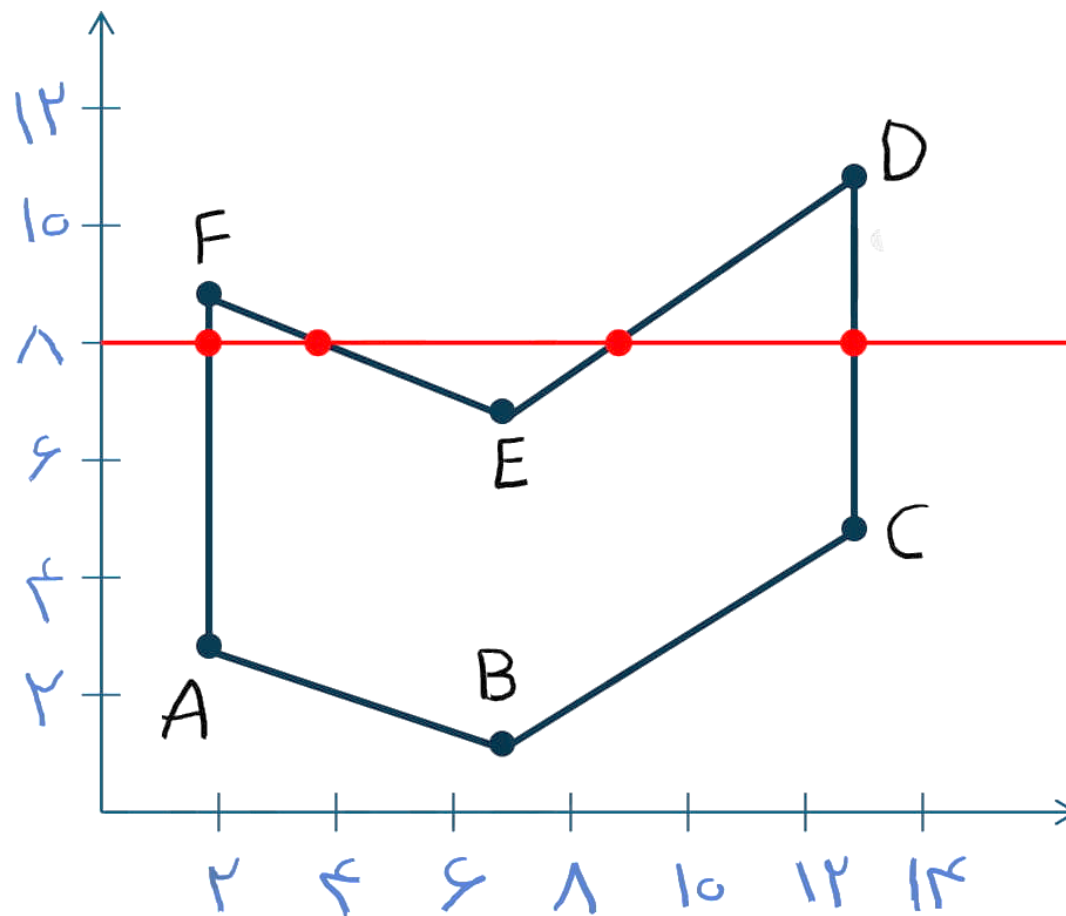
نقاط تقاطع را بر حسب مؤلفه‌ی x به صورت صعودی مرتب می‌کند.

بین هر جفت نقطه در ترتیب فوق را با پیکسل پر می‌کند.

پر کردن ناحیه



پر کردن ناحیه



$$\text{نقاط تقاطع} = (2, 4, 9, 13) \Rightarrow (2, 4) \text{ و } (9, 13)$$

پر کردن ناحیه

حالت‌های خاص ۱:



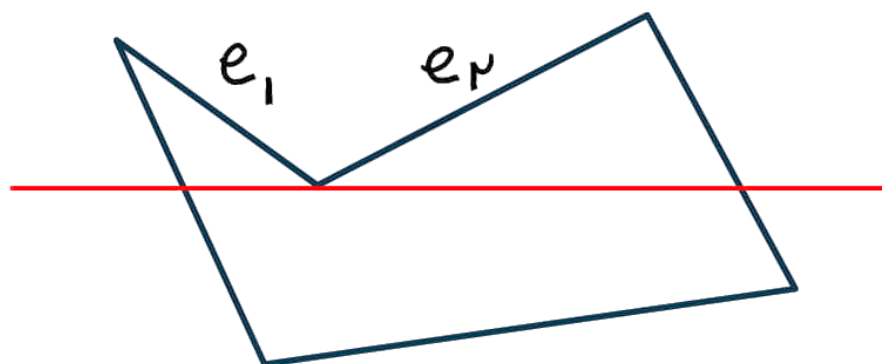
پر کردن ناحیه

حالت‌های خاص ۱:



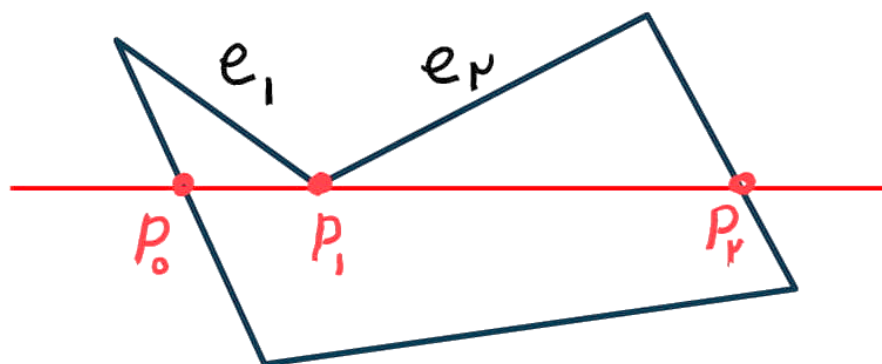
پر کردن ناحیه

حالت‌های خاص ۱:



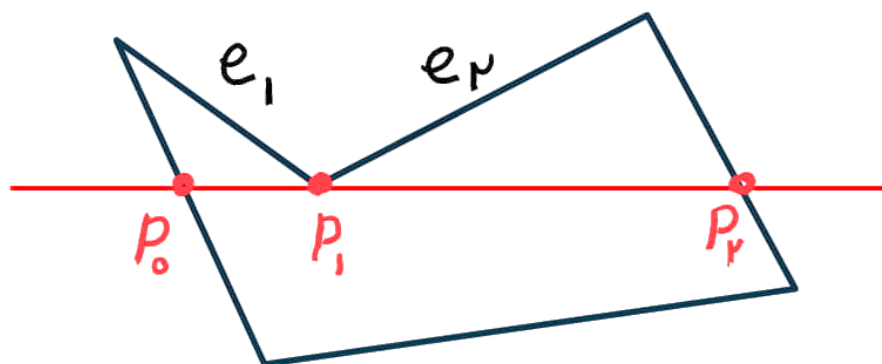
پر کردن ناحیه

حالت‌های خاص ۱:



پر کردن ناحیه

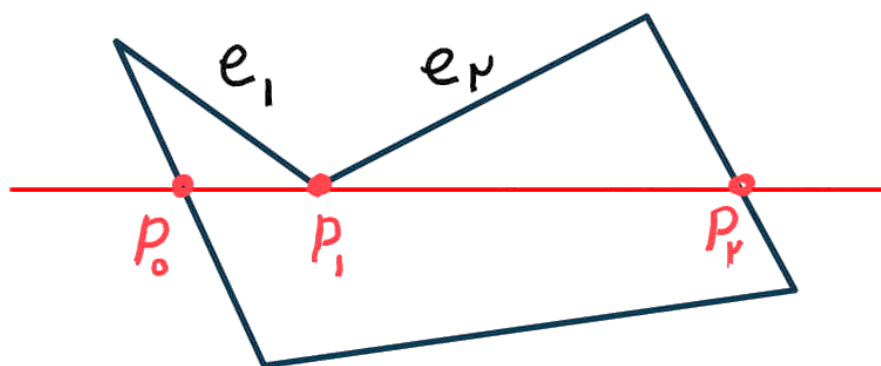
حالت‌های خاص ۱:



$$نقاط تقاطع = (P_0, P_1, P_2) \Rightarrow ?$$

پر کردن ناحیه

راه حل:

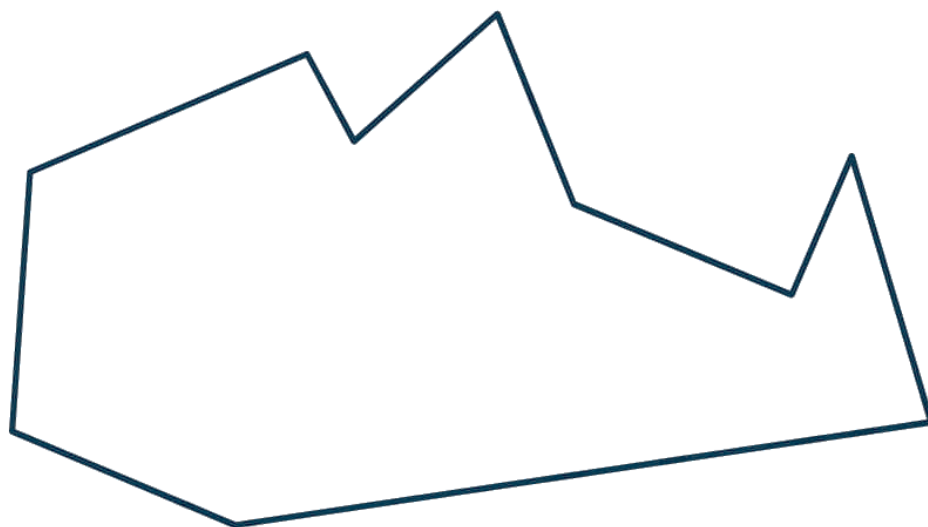


نقطه‌ی P_1 هم به یال e_1 و هم به یال e_2 مربوط است. بنابراین دوباره آن را تکرار می‌کنیم:

$$(P_0, P_1, P_1, P_2) \Rightarrow (P_0, P_1), (P_1, P_2)$$

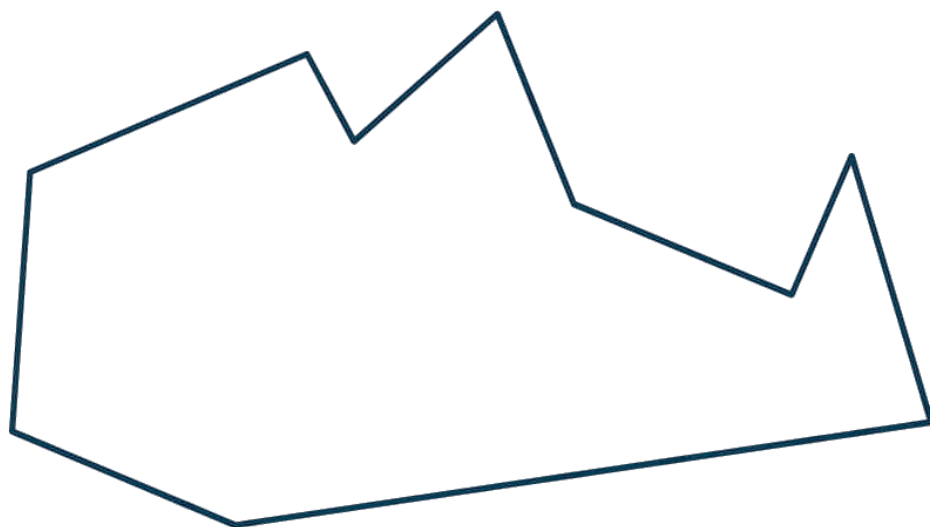
پر کردن ناحیه

حالت‌های خاص ۲:



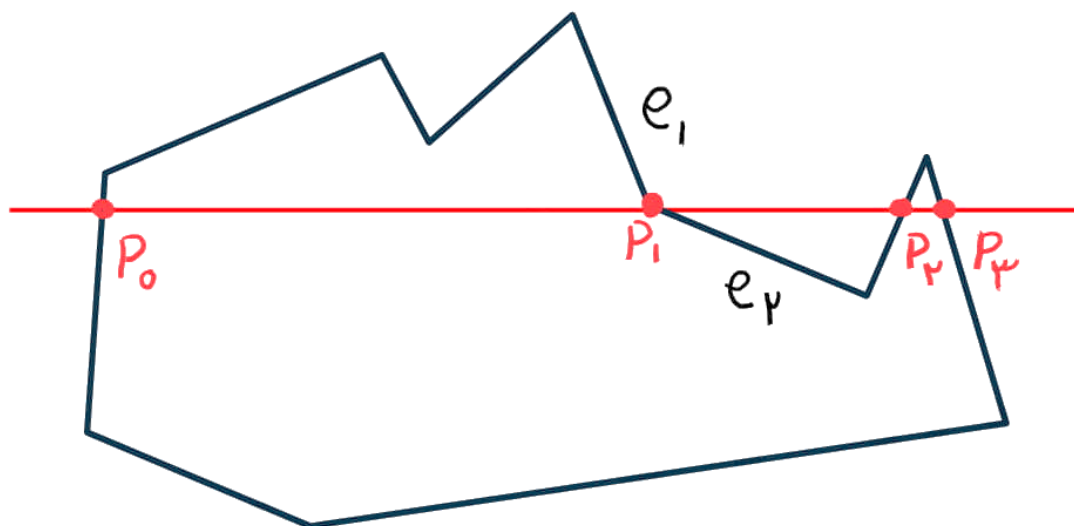
پر کردن ناحیه

حالت‌های خاص ۲:



پر کردن ناحیه

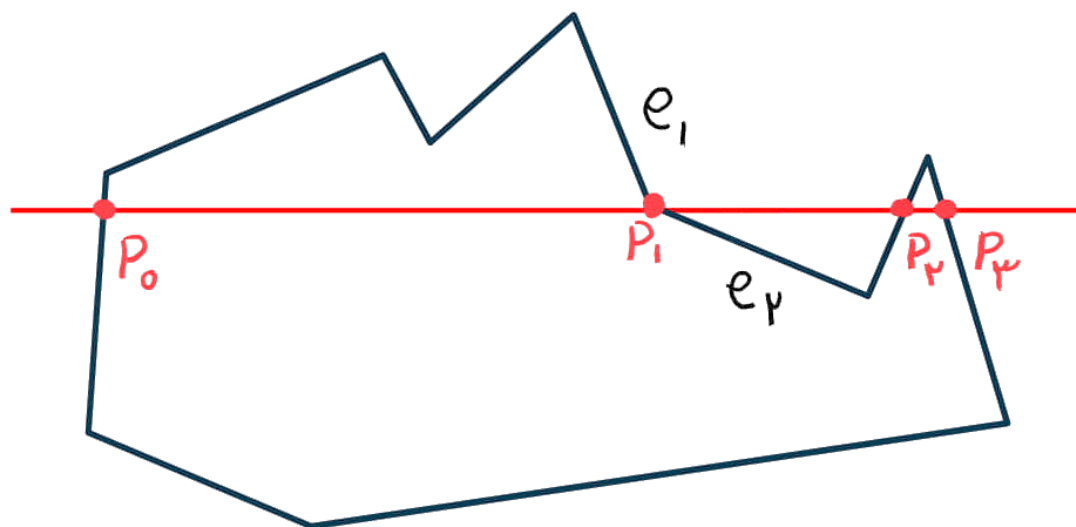
حالت‌های خاص ۲:



در این حالت نمی‌توان P_1 را دو بار تکرار کرد، زیرا در این صورت فاصله‌ی بین P_1 و P_2 پر می‌شود که صحیح نیست و خارج از چندضلعی است.

پر کردن ناحیه

راه حل:



اگر نقطه‌ی تقاطع دو یال ماکزیمم یا مینیمم موضعی باشد آن را دو بار تکرار می‌کنیم.
در غیر این صورت یک بار نوشتن نقطه‌ی مورد نظر کافی است.

پر کردن ناحیه

نکته ۱:

از یال‌های افقی در این الگوریتم صرف‌نظر می‌شود، زیرا پیکسل‌های آن به طور خودکار در تبدیل اسکن پر می‌شوند.

نکته ۲:

برای پیاده‌سازی این الگوریتم نیاز به محاسبات فراوان می‌باشد، ولی مزیت آن نسبت به دو الگوریتم قبل، توانایی در پر کردن اشکال چندضلعی نامنتظم است.

پُر کردن ناحیه

تولید کاراکتر

یکی از مباحث مهم در بحث تبدیل اسکن، تولید یک کاراکتر است.

منظور از یک کاراکتر حروف، اعداد، نمادهای نوشتاری و سمبل‌های دیگری است که قرار است به سائزها و فرم‌های مختلفی ظاهر شوند.

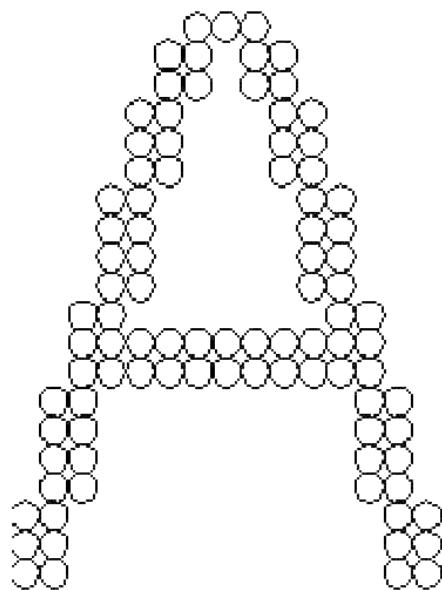
طرح کلی یک مجموعه از کاراکترها را **فونت** (Font) می‌نامند.

پر کردن ناحیه

تولید کاراکتر

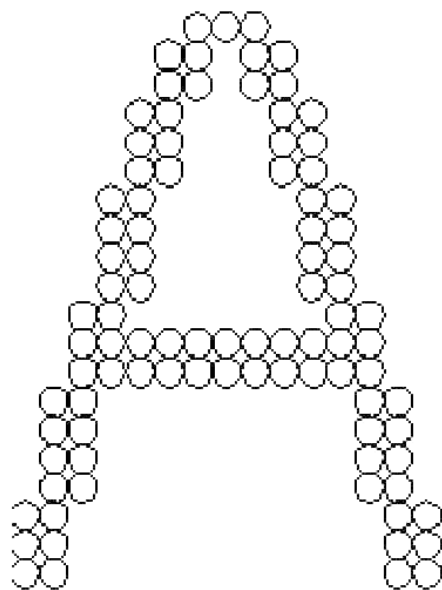
به طور کلی دو روش برای ذخیره نمودن فونت‌ها به کار می‌رود:

روش اول که روش راحت‌تری است استفاده از **طرح شبکه‌ای** (Grid Pattern) یا Bitmap است.



پر کردن ناحیه

طرح شبکه ای



یعنی کاراکتر را در یک طرح شبکه‌ای نشان می‌دهند.

در این روش یک شبکه‌بندی برای تمام کاراکترها وجود دارد.

پر کردن ناحیه

طرح شبکه ای

از مزیت‌های این روش سرعت واقعی اجرای آن است.

از معایب این روش این است که فونت‌های Bitmap نیاز به فضای زیادی دارند.

زیرا هر تغییری از آن (از لحاظ سائز و فرمت) معمولاً در یک شبکه‌ی جداگانه ذخیره می‌شود.

پر کردن ناحیه

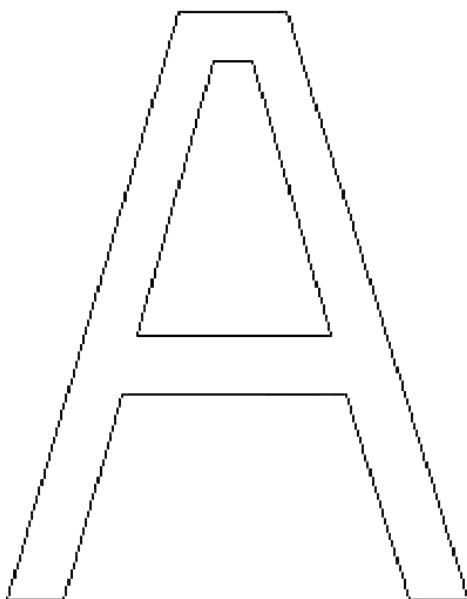
طرح شبکه ای

اگر بخواهیم تغییراتی در فونت‌های Bitmap داشته باشیم، مثل دوران یک حرف، فونت‌های Bitmap خیلی محدود هستند.

دوران ۱۸۰ درجه خیلی ساده است اما دوران به اندازه‌ی هر زاویه‌ی دلخواه نیاز به دوباره مدل‌سازی حروف بر روی شبکه‌ی جدید خواهد داشت.

پر کردن ناحیه

تولید کاراکتر



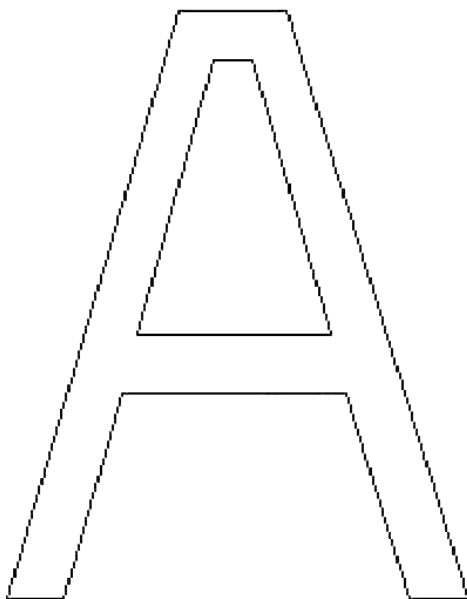
روش دوم نسبت به روش اول انعطاف پذیرتر است.

فونت‌ها در این روش توسط **خطوط بیرونی فونت**

(Outline Font) ذخیره می‌شوند.

پر کردن ناحیه

تولید کاراکتر



با این روش شکل یک کاراکتر با استفاده از یک سری خطوط
راست و منحنی‌ها نمایش داده می‌شوند.

سپس از یکی از الگوریتم‌های پر کردن ناحیه، برای پر کردن
سطح کاراکتر استفاده می‌شود.

پر کردن ناحیه

خطوط بیرونی فونت

از مزایای این روش، استفاده‌ی کمتر آن از فضا برای ذخیره‌ی کاراکتر است
از معایب آن، زمان زیاد اجرای آن است.